Algoritmo e Laboratório de Programação II

Aula 6 – Ponteiros - parte 2 (Nível Avançado)

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Ponteiros no Lugar de Matrizes

- Em C, o relacionamento entre ponteiros e matrizes é tão estreito que estes deveriam ser realmente tratados juntos.
- O compilador transforma matrizes em ponteiros, pois a arquitetura do microcomputador compreende ponteiros, e não matrizes.
- Qualquer operação que possa ser feita com índices de uma matriz pode ser feita com ponteiros.

Ponteiros no Lugar de Matrizes

- Na aula de matrizes, aprendemos que o nome de uma matriz representa seu endereço de memória.
- Esse endereço é o do primeiro elemento da matriz.
- Em outras palavras, o nome de uma matriz é um ponteiro que aponta para o primeiro elemento da matriz.
- Para esclarecer a relação entre ponteiros e matrizes, vamos examinar:

Ponteiros no Lugar de Matrizes

Prog8.c

```c
/* Prog8.c */
/* Imprime os elementos de uma matriz usando notação matriz */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    static int M[5]= {92,81,70,69,58};
    int i;
    for(i=0; i<5; i++)
        printf("%d\n", M[i]); /* Notação matriz */

    system("PAUSE");
    return 0;
}
```

Ponteiros no Lugar de Matrizes

- O próximo exemplo usa a notação ponteiro:

Prog9.c

```c
/* Prog9.c */
/* Imprime os elementos de uma matriz usando notação ponteiro */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    static int M[5]= {92,81,70,69,58};
    int i;
    for(i=0; i<5; i++)
        printf("%d\n", *(M + i)); /* Notação ponteiro */

    system("PAUSE");
    return 0;
}
```

Ponteiros no Lugar de Matrizes

- A expressão *(M + i) tem exatamente o mesmo valor de M[i].
- Você já sabe que M é um ponteiro int e aponta para M[0], conhecendo também a aritmética com ponteiros.
- Assim, se somarmos 1 a M, obteremos o endereço de M[1]; M + 2 é o endereço de M[2] e assim por diante.

Ponteiros no Lugar de Matrizes

- Em regra geral, temos que:

```
M + i é equivalente a &M[i], portanto
    *(M + i) é equivalente a M[i]
```

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Ponteiros Constantes e Ponteiros Variáveis

- Analisando o exemplo anterior, você poderia perguntar se a instrução

```
printf("%d\n", *(M + i)); /* Notação ponteiro */
```

- Não poderia ser simplificada por

```
printf("%d\n", *(M ++)); /* ERRO */
```

- A resposta é não.
- A razão disso é que não podemos incrementar uma constante.

Ponteiros Constantes e Ponteiros Variáveis

- Da mesma forma que existem inteiros constantes e inteiros variáveis, existem ponteiros constantes e ponteiros variáveis.
- O nome de uma matriz é um ponteiro constante.
- Isso vale para qualquer constante.
- Você não poderia escrever:

```
x = 3++; /* ERRADO */
```

- O compilador apresentaria um erro.

Ponteiros Constantes e Ponteiros Variáveis

- O nome de uma matriz é um ponteiro constante e não pode ser alterado.
- Um ponteiro variável é um lugar na memória que armazena um endereço.
- Um ponteiro constante é um endereço, uma simples referência.
- Vamos reescrever o programa anterior usando um ponteiro variável em vez do nome da matriz.

Ponteiros Constantes e Ponteiros Variáveis

Prog10.c

```c
/* Prog10.c */
/* Imprime os elementos de uma matriz usando notação ponteiro */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    static int M[5]= {92,81,70,69,58};
    int i, *p = M; /* cria e inicializa o ponteiro variável */

    for(i=0; i<5; i++)
        printf("%d\n", *(p++)); /* Notação ponteiro */

    system("PAUSE");
    return 0;
}
```

Ponteiros Constantes e Ponteiros Variáveis

- Nessa versão, definimos um ponteiro para um int e o inicializamos com o nome da matriz: `int *p = M;`
- Agora podemos usar p em todo lugar do programa que usa M e, como p é um ponteiro variável e não uma constante, podemos usar expressões como:

```
*(p++)
```

Ponteiros Constantes e Ponteiros Variáveis

- O ponteiro p contém inicialmente o endereço do primeiro elemento da matriz &M[0].
- Para acessar o próximo elemento, basta incrementar p de um.
- Após o incremento, p aponta para o proximo elemento, &M[1], e a expressão *(p++) representa o conteúdo desse segundo elemento.
- O laço for faz com que a expressão acesse cada um dos elementos em ordem.

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Passando Matrizes como Argumento para Funções

- Na aula sobre matrizes, apresentamos numerosos exemplos de como elas são passadas como argumentos para funções e como as funções podem acessar seus elementos.
- Quando uma função recebe o endereço de uma matriz como argumento, ela o declara usando o nome do tipo e colchetes ([]).

Passando Matrizes como Argumento para Funções

- Essa notação declara ponteiros constantes e não ponteiros variáveis.
- Entretanto, é mais conveniente usar a notação ponteiro no lugar da notação matriz.
- A notação ponteiro declara um ponteiro variável.
- O próximo exemplo modifica o programa que calcula a média para que use ponteiros no lugar de matriz.

Passando Matrizes como Argumento para Funções

Prog11.c

```c
/* Prog11.c */
/* Mostra passagem de matrizes para funções usando ponteiros */
#include <stdio.h>
#include <stdlib.h>
#define TAMANHO 50

float media(float *, int);/* protótipo */

int main()
{
    float notas[TAMANHO] , m;
    int i=0;

    do
    {
        printf("Digite a nota do aluno %d ", i+1);
        scanf("%f", notas + i);
    }
    while( *(notas + i++) >= 0.0);

    i--; /* remove o item de término */

    m = media( notas, i );

    printf("Média das notas: %.2f\n", m);

    system("PAUSE");
    return 0;
}
```

Passando Matrizes como Argumento para Funções

```
31  /* Calcula a média dos valores da matriz */
32  float media(float *lista, int tamanho)
33  {
34      int i;
35      float m=0.0;
36      for(i=0; i < tamanho ; i++) m += *(lista++);
37      return  m/tamanho ;
38  }
```

Passando Matrizes como Argumento para Funções

- Observe primeiramente o protótipo da função media().

```
float media(float *, int);/* protótipo */
```

- A declaração: `float *` é equivalente à original

```
float[]
```

- A primeira declara um ponteiro variável, enquanto a segunda declara um ponteiro constante.

Passando Matrizes como Argumento para Funções

- Como o nome da matriz é um endereço, não usamos o operador de endereços (&) na instrução de chamada à função:

```
m = media( notas, i );
```

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Ponteiros e Strings

- Strings são matrizes do tipo char.
- Dessa forma, a notação ponteiro pode ser aplicada.
- Como primeiro exemplo, vamos escrever uma função que procure um caractere numa cadeia de caracteres.
- Essa função retorna o endereço da primeira ocorrência do caractere, se este existir, ou o endereço zero, caso o caractere não seja encontrado.

Ponteiros e Strings

Prog12.c

```c
/* Prog12.c */
/* Procura um caractere numa cadeia de caracteres */
#include <stdio.h>
#include <stdlib.h>

char * procura(char *, char );/* protótipo */

int main()
{
    char str[81], *ptr;

    printf("Digite uma frase:\n");
    gets(str);

    ptr = procura(str, 'h');

    printf("\nA frase começa no endereço %p\n", str);

    if(ptr)
    {
        printf("\nPrimeira ocorrência do caractere 'h': %p\n", ptr);
        printf( "\nA sua posição é: %d\n", ptr-str);
    }
    else
        printf( "O caractere 'h' não existe nesta frase.\n");

    system("PAUSE");
    return 0;
}
```

Ponteiros e Strings

```
31  /* Procura um caractere numa frase */
32  char *procura(char *s, char ch)
33  {
34      while( *s != ch && *s != '\0') s++;
35      if(*s != '\0') return s;
36      return (char *)0;
37  }
```

Ponteiros e Strings

- Vamos analisar o protótipo da função procura().

```
char * procura(char *, char );/* protótipo */
```

- A função retorna um ponteiro char.
- Assim, funções podem tanto receber ponteiros como argumentos ou retornar um ponteiro, desde que declarados de acordo.

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Funções de Biblioteca para Manipulação de Strings

- Você já usou várias funções de biblioteca para manipular strings.
- Essas funções usam ponteiros.
- Apesar de existirem prontas na biblioteca C, escrevemos algumas delas para que você aprenda e as crie conforme suas necessidades.

Funções de Biblioteca para Manipulação de Strings

Prog13.c

```
1  /* Prog13.c */
2  /* Funções para Manipular Strings */
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  /* Protótipos */
7  int strlen(char *);
8  void strcpy(char *, char *);
9  int strcmp(char *, char *);
10
11 int main()
12 {
13     int len, i;
14     char nome[80];
15     char str[] = "Carrta";
16     char resposta[]="BRANCO";
17     char resp[40];
18
19     printf("Digite o seu nome: ");
20     gets(nome);
21     len=strlen(nome);
22     for(i=0; i<len + 4; i++)
23         printf("Endereço = %u\tChar  = %c\tDec   = %d\n",
24                 (unsigned)(nome+i), nome[i], (int)nome[i]);
```

Funções de Biblioteca para Manipulação de Strings

```
26      printf("\n\n%s\n", str);
27      strcpy(str+2, str+3); // apaga um caractere da cadeia
28      printf("%s\n\n", str);
29
30      printf("\nQual é a cor do cavalo branco de Napoleão?");
31      gets(resp);
32
33      while (strcmp(resp,resposta) != 0)
34      {
35          printf("Resposta errada. Tente de novo.\n");
36          gets(resp);
37      }
38      printf("Correto!\n");
39      return 0;
40
41  }
```

Funções de Biblioteca para Manipulação de Strings

```c
/* Retorna o tamanho da cadeia */
int strlen(char *s)
{
    int i = 0;
    while (*(s++)) i++;
    return i;
}

/* Copia a cadeia origem na cadeia destino */
void strcpy(char *dest, char *orig)
{
    while (*(dest++) = *(orig++));
}

/* Compara a cadeia s com a cadeia t
*  Retorna a diferença ASCII:
*       um número positivo se s > t
*       um número negativo se s < t
*       zero se s == t
*/
int strcmp(char *s, char *t)
{
    while(*s == *t && *s)
    {
        s++;
        t++;
    }
    return *s - *t;
}
```

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Ponteiros para uma Cadeia de Caracteres Constante

- Vamos analisar duas maneiras de inicializar cadeias de caracteres constantes:
  - Usando um ponteiro constante;
  - Usando um ponteiro variável.
- Observe o exemplo:
- O ponteiro constante poderia ser usado em situações em que não queremos alterar o conteúdo da cadeia de caracteres, como para imprimir, enviar como argumento para uma função etc.

Ponteiros para uma Cadeia de Caracteres Constante

Prog14.c

```c
/* Prog14.c */
/* Mostra o uso de ponteiro para cadeias de caracteres */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    char s1[] = "Saudações!";
    char *s2 = "Saudações!";

    printf("%p\n", s1);
    printf("%p\n", s2);

    s1++; /* ERRO. Não podemos incrementar uma constante */
    s2++; /* OK */

    printf("%s\n", s2); /* Imprime: Saudações */
    return 0;
}
```

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Matrizes de Ponteiros

- Matrizes de ponteiros são essencialmente utilizadas para substituir matrizes de duas dimensões em que cada elemento é uma cadeia de caracteres.
- Esse uso permite uma grande economia de memória, já que não teremos a desvantagem de dimensionar todos os elementos com o tamanho da maior cadeia.

Matrizes de Ponteiros

- Vamos modificar um programa da aula de matrizes, para que utilize uma matriz de ponteiros no lugar de uma matriz de strings.

Matrizes de Ponteiros

Prog15.c

```
1  /* Prog15.c */
2  /* Imprime o dia da semana a partir de uma data */
3  /* Mostra o uso de uma matriz de ponteiros */
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <conio.h>  /* para getche() */
8
9  int dsemana(int, int, int); /*Protótipo */
10
11 int main()
12 {
13     static char *diasemana[7]=
14     {
15         "Domingo",
16         "Segunda-feira",
17         "Terça-feira",
18         "Quarta-feira",
19         "Quinta-feira",
20         "Sexta-feira",
21         "Sábado"
22     };
```

Matrizes de Ponteiros

```
23      int dia, mes, ano;
24      const char ESC = 27;
25      do
26      {
27          printf("Digite a data na forma dd mm aaaa: ");
28          scanf("%d%d%d", &dia, &mes, &ano);
29          printf("%s\n", diasemana [ dsemana(dia,mes,ano)] );
30          printf("ESC para terminar ou ENTER para recomeçar\n");
31      }
32      while (getch() != ESC);
33
34      system("PAUSE");
35      return 0;
36  }
```

Matrizes de Ponteiros

```
38  /* Encontra o dia da semana a partir de uma data
39   * Retorna 0 para domingo, 1 para segunda-feira etc.
40   */
41  int dsemana(int dia, int mes, int ano)
42  {
43      int dSemana = ano + dia + 3 * (mes - 1) - 1;
44      if( mes < 3)
45          ano--;
46      else
47          dSemana -= (int)(0.4*mes+2.3);
48      dSemana += (int)(ano/4) - (int)((ano/100 + 1)*0.75);
49      dSemana %= 7;
50      return dSemana;
51  }
```

Matrizes de Ponteiros

- Na versão matriz, as cadeias de caracteres são guardadas na memória em sete posições de 14 bytes cada uma, ou seja, ocupando 98 bytes de memória.
- Na nova versão, as cadeias são guardadas de forma a ocupar somente o número de bytes necessários para o seu armazenamento.
- Cada elemento da matriz é um ponteiro e não mais outra matriz.

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Matriz de Strings e a Memória Alocada

- A versão do código que usa matriz de string aloca 98 bytes de memória da seguinte forma:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
1	D	o	m	i	n	g	o	\0						
2	S	e	g	u	n	d	a	-	f	e	i	r	a	\0
3	T	e	r	ç	a	-	f	e	i	r	a	\0		
4	Q	u	a	r	t	a	-	f	e	i	r	a	\0	
5	Q	u	i	n	t	a	-	f	e	i	r	a	\0	
6	S	e	x	t	a	-	f	e	i	r	a	\0		
7	S	á	b	a	d	o	\0							

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Matriz de Ponteiros e a Memória Alocada

- A versão do código que usa ponteiros aloca 79 bytes de memória da seguinte forma:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13
0	D	o	m	i	n	g	o	\0	S	e	g	u	n	d
1	a	-	f	e	i	r	a	\0	T	e	r	ç	a	-
2	f	e	i	r	a	\0	Q	u	a	r	t	a	-	f
3	e	i	r	a	\0	Q	u	i	n	t	a	-	f	e
4	i	r	a	\0	S	e	x	t	a	-	f	e	i	r
5	a	\0	S	á	b	a	d	o	\0					

- A versão que usa ponteiros aloca uma matriz de sete ponteiros e inicializa cada um deles com o endereço de cada uma das cadeias de caracteres constantes.

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Ponteiros para Ponteiros

- No próximo programa, mostraremos a ordenação de nomes por meio de uma matriz de ponteiros.
- No lugar de ordenar os próprios nomes, ordenaremos a matriz de ponteiros em que cada elemento aponta para um dos nomes.
- Os elementos da matriz de ponteiros são ponteiros que apontam para outros ponteiros
- Veja a listagem:

Ponteiros para Ponteiros

Prog16.c

```c
/* Prog16.c */
/* Mostra o uso de ponteiros para ponteiros */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define NOME_MAX 30
#define TAM_MAX 100

void ordena(char **p,int n)
{
    char *temp;
    int i,j;

    for(i=0; i<n-1; i++)
    {
        for(j=i+1; j<n; j++)
            if( strcmp(p[i],p[j])>0)
            {
                temp = *(p+i);
                *(p+i) = *(p+j);
                *(p+j) = temp;
            }
    }
}
```

Ponteiros para Ponteiros

```
26  int main()
27  {
28      char nomes[NOME_MAX][TAM_MAX];
29      char *p[30]; /* matriz de ponteiros */
30      int n, i;
31
32      for( n=0;; n++)
33      {
34          printf("Digite nome ou [ENTER] para fim: ");
35          gets(nomes[n]);
36          if(strlen(nomes[n])==0) break;
37          p[n] = nomes[n];
38      }
39
40      printf("\n\nLista original:\n");
41      for(i=0; i<n; i++)
42          printf("%s\n", p[i]);
43
44      ordena(p,n);
45
46      printf("\n\nLista ordenada:\n");
47      for(i=0; i<n; i++)
48          printf("%s\n", p[i]);
49
50      system("PAUSE");
51      return 0;
52  }
```

Ponteiros para Ponteiros

- O programa declara uma matriz de duas dimensões para armazenar os nomes, em seguida cria uma matriz de ponteiros que apontarão para eles.
- Toda vez que o usuário digitar um nome, seu endereço é atribuído a um elemento da matriz de ponteiros.
- Quando [ENTER] for digitado, o processo é interrompido.

Ponteiros para Ponteiros

- Os nomes são impressos duas vezes:
  - A primeira, na mesma ordem em que foram digitados;
  - E a segunda, em ordem alfabética.

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Ordenando Ponteiros

- A novidade desse programa está em como os nomes são ordenados.
- Na realidade, a função ordena() não ordena os nomes, e sim os ponteiros para os nomes.
- Essa ordenação é muito mais rápida que a da própria matriz nomes[][], pois estamos movimentando ponteiros pela memória, e não rearranjando cada letra de cada nome.

Ordenando Ponteiros

- Os ponteiros são variáveis que ocupam pouco lugar de memória, entretanto os nomes podem ocupar muita memória.
- Observe a declaração do primeiro argumento de ordena(): `void ordena(char **p,int n)`
- O tipo da variável p é char**.
- Essa notação indica que p é um ponteiro duplamente indireto.

Ordenando Ponteiros

- Quando o endereço de um nome é passado para uma função como argumento, você já sabe que o seu tipo é char*.
- Um único asterisco é usado para indicar o endereço de um único nome.
- A função ordena() não recebe o endereço de um nome, mas o de uma matriz de ponteiros para nomes.
- O nome de uma matriz é um ponteiro para um elemento dela.

Ordenando Ponteiros

- No caso da nossas matriz, um elemento é um ponteiro.
- Portanto, o nome da matriz é um ponteiro que aponta para outro ponteiro.
- Dois asteriscos são usados para indicar um ponteiro para ponteiro.

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

Notação Ponteiro para Matrizes de Ponteiros

- Cada elemento da matriz p é um ponteiro para um nome.
- Assim p[i] é o endereço de um nome.
- Você já sabe que um elemento de uma matriz pode ser escrito em notação ponteiro.
- Dessa forma, podemos escrever a expressão p[i] como:

```
*(p+i)
```

Roteiro

- Ponteiros no lugar de matrizes
- Ponteiros constantes e ponteiros variáveis
- Passando matrizes como argumento para funções
- Ponteiros e string
- Funções de biblioteca para manipulação de strings
- Ponteiros para uma cadeia de caracteres constante
- Matrizes de ponteiros
- Matriz de strings e a memória alocada
- Matriz de ponteiros e a memória alocada
- Ponteiros para ponteiros
- Ordenando ponteiros
- Notação ponteiro para matrizes de ponteiros
- O programa do foguete

O Programa do Foguete

- Vamos analisar um exemplo que incorpora acesso duplamente indireto e mostra como manipular elementos de uma matriz usando ponteiros que apontam para ponteiros.
- Este programa modela um foguete e imprime-o na tela.

O Programa do Foguete

- O programa consiste em um amplo laço.
- Cada vez que o laço é executado, o programa desenha o foguete e uma linha, representando a terra, composta de cinco caracteres de linha dupla (‘\xCD’).
- O foguete é definido por quatro caracteres: o corpo (‘\xDB), o cone que forma o bico do foguete (‘\x1E’) e duas máquinas (‘\x1E’).

O Programa do Foguete

- Depois de desenhado o foguete, as linhas da matriz que o contém são rodadas;
- Cada linha é movida uma posição acima e a primeira linha é colocada no lugar da última (na verdade, a matriz não é mexida; movemos uma matriz de ponteiros para as linhas).
- A impressão é executada novamente.
- O foguete, então, se distância da Terra.

O Programa do Foguete

Prog17.c

```c
/* Prog17.c */
/* movimenta imagem na tela */
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#define LIN 10
#define COL 5

int main()
{
    int cont, j, k;
    char *ptr[LIN];
    char *temp;

    static char foguet[LIN][COL] =
    {
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, 0, 0, 0},
        { 0, 0, '\x1E' , 0, 0} ,
        { 0, '\x1E','\xDB', '\x1E',0 }
    };
```

O Programa do Foguete

```
29      static char terra[]=
30      { '\xCD' ,'\xCD', '\xCD' ,'\xCD' ,'\xCD' ,'\0'};
31
32      for(cont=0; cont<LIN; cont++)
33          *(ptr+cont)=*(foguet+cont);
34      for(cont=0; cont<LIN-1; cont++)
35      {
36          for(j=0; j<LIN ; j++)
37          {
38              for(k=0; k<COL; k++)
39                  printf("%c", *(*(ptr+j)+k));/* ptr[j][k] */
40              printf("\n");
41          }
42          printf("%s\n",terra);
43          getch();
44          system("cls"); /* Limpa a tela */
45          temp=*ptr;
46          for(j=0; j<LIN-1; j++)
47              *(ptr+j) = *(ptr+j+1);
48          *(ptr+LIN-1)=temp;
49      }
50      system("pause");
51      return 0;
52  }
```

O Programa do Foguete

- Declaramos uma matriz de ponteiros para armazenar as linhas do desenho do foguete.

```
char *ptr[LIN];
```

- A cada elemento da matriz, atribuímos o endereço de uma linha da matriz foguet[][] usada no laço:

```
for(cont=0; cont<LIN; cont++)
    *(ptr+cont)=*(foguet+cont);
```

O Programa do Foguete

- Os elementos da matriz são impressos usando dois laços aninhados.
- Foi usada notação de ponteiro para referenciar um elemento individual da matriz:

```
printf("%c", *(*(ptr+j)+k));/* ptr[j][k] */
```

- Aqui nós referenciamos propriamente a matriz foguet[][], mas sim a matriz de ponteiros ptr[], que aponta para ela.

O Programa do Foguete

- Para movimentar o foguete, se tivéssemos de mover os elementos de cada linha da matriz para cima, necessitaríamos mover 50 caracteres na memória, o que consumiria muito tempo.
- Em vez disso, simplesmente movemos os ponteiros que apontam para as linhas.
- Teremos, então, o movimento de dez ponteiros, o que torna a operação bem mais rápida.

O Programa do Foguete

- É feita uma rotação com os ponteiros da matriz ptr[], cada elemento é movido uma localização acima, e o elemento do topo é movido para o fim.
- O primeiro passo é guardar o valor do topo ptr[0] na variável temp, então usar um laço para mover o conteúdo de cada ponteiro para o imediatamente anterior e, finalmente, o elemento de topo, guardado na variável temp, é inserido.

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;